

宁波大学
NINGBO UNIVERSITY

网络集成实践

实验设计报告

姓名：	顾凯琪
学号：	226001624
课程名称：	网络集成实习
所在学院：	信息科学与工程学院
专业班级：	22级计算机网络工程方向
任课教师：	陈海明

实验名称	网络系统服务-Web 服务创建

一、 实验的目的与要求：

用 C++ Socket 编程实现一个 Web 服务器（端口号为 8080 或者其它）

实验要求：

1. 该 Web 服务器在一台主机上运行，支持多台主机同时访问（多线程）
2. 有服务开启和关闭的开关
3. 能够返回指定主页（index.html），页面可以是静态的内容，也可以包括实现 HTTP 的四个操作的动态交互

测试方法：

在同组其他主机上，打开浏览器，输入 http://主机 IP 地址：8080 或指定的端口号，浏览器上显示主页

完成实验报告，要求给出实现的方法的具体说明和测试结果

二、 实验的设备及软件：

2.1 实验设备：

1. 计算机 若干

2.2 实验软件：

1. C++开发环境
2. 浏览器
3. Postman 等工具

三、 系统设计的原理和流程图：

3.1 实验原理分析

本实验旨在通过 C++ 的 Socket 编程实现一个基础的 Web 服务器，支持同时处理多个客户端的并发请求，并能向客户端返回指定的网页内容。服务器端采用 winsock2.h 提供的套接字接口，完成套接字的创建、绑定、监听及连接接收等核心操作，实现与客户端的网络通信。在并发处理方面，服务器使用多线程技术，为每个客户端连接分配独立的线程，以保证多个客户端请求能够被同时响应，提高服务器的并发处理能力和响应效率。

服务器启动后将在指定端口监听来自客户端的连接请求，并在接收到请求后，解析客户端发出的 HTTP 请求报文，生成符合 HTTP 协议的响应报文，并返回相应的静态网页内容（如 index.html 文件）给客户端。服务器程序最终被编译为一个可执行文件，运行该程序即可在指定端口开启服务，通过结束该进程实现服务器的关闭。整个过程涵盖了网络编程、线程管理以及 HTTP 协议的基本实现，为后续开发更复杂的网络服务系统打下基础。

3.2 实验流程图

综上可得，实验代码的大致包括以下若干模块。

1. 服务器初始化阶段：通过调用 WSAStartup() 初始化 Windows Socket 库，随后创建 TCP 套接字，指定使用 IPv4 和流式传输方式。接着将套接字绑定到本地任意 IP 地址和指定端口（如 9999），并通过 listen() 函数设置监听状态，等待客户端发起连接请求。
2. 客户端连接处理：服务器在主循环中使用 accept() 函数接受客户端的连接请求，成功连接后为每个客户端创建一个独立的通信套接字，便于后续的通信操作。
3. HTTP 请求解析与响应生成：服务器接收到客户端的数据后，将其解析

为 HTTP 请求报文，并根据请求内容判断所请求的资源路径。如果资源存在（如 mail.html 文件），则调用 sendHtml() 函数逐行读取 HTML 内容并发送给客户端；如果文件不存在，则可扩展处理逻辑返回 404 Not Found 响应。

4. 数据读取与响应发送：在发送响应前，服务器还可构造 HTTP 响应头部信息，并通过 send() 函数将其发送至客户端，随后将请求资源的正文部分发送完成，并在传输结束后关闭该客户端连接。
5. 服务端循环处理与资源回收：整个服务运行在一个无限循环中，持续接收并处理新的客户端请求。每处理完一个客户端连接后，通过 closesocket() 关闭对应连接，并在服务器退出前通过 WSACleanup() 清理 Winsock 环境。

实验代码的大致流程图如图 3.1 所示。

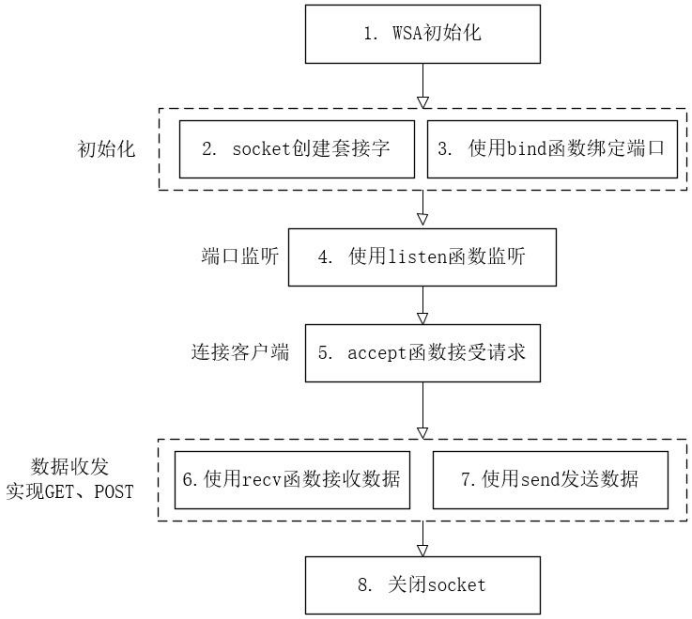


图 3.1 实验代码的大致流程图

四、 系统实现的过程、步骤及测试结果

4.1 代码设计

按照图 3.1 所示步骤依次进行代码编写，采用模块化形式进行代码的编写。

（1）辅助函数与网络库初始化

为了启用 Windows 系统下的网络编程接口，程序首先定义了 initSocket() 函数对 Winsock 网络库进行初始化。该函数通过 MAKEWORD(2,2) 指定使用 2.2 版本的 Winsock，随后调用 WSAStartup 函数传入版本号和一个 WSADATA 结构体变量作为参数以完成初始化。如果初始化失败，则调用自定义的错误处理函数 error_die() 报错并退出程序。

error_die() 是一个错误处理辅助函数，接受一个字符串作为提示信息，并调用 WSAGetLastError() 获取最近一次的 Winsock 错误码，最后通过 printf 输出错误信息并终止程序。该函数在本程序中被频繁调用，用于处理诸如 socket 创建失败、绑定失败、接受连接失败等网络编程中的常见错误，是提升调试效率和增强健壮性的关键组成。

```
1. void error_die(const char*str)
2. {
3.     printf("[hint]%s failed:%d",str,WSAGetLastError());
4.     exit(-1);
5. }
6.
7. void initSocket()
8. {
9.     WORD version = MAKEWORD(2,2);
10.    WSADATA wsadata;
11.    if (WSAStartup(version, &wsadata) != 0)
12.    {
13.        error_die("WSAStartup");
14.    }
15.}
```

（2）服务器套接字的创建与监听

服务器通过 listenClient() 函数创建用于监听客户端请求的 socket。首先，调用 socket(AF_INET, SOCK_STREAM, IPPROTO_TCP) 创建一个基于 IPv4、面向连接的 TCP socket，如果创建失败则调用 error_die() 报错。随后，使用 sockaddr_in 结构体设定本地绑定信息，其中 sin_family 设置为 AF_INET 表示 IPv4，sin_port 使用 htons(9999) 将本地端口 9999 转为网络字节序，sin_addr.S_un.S_addr 设置为 INADDR_ANY，表示绑定本机所有可用 IP 地址。

接下来，调用 bind() 将该 socket 绑定到指定的本地地址和端口，如果绑定失败则立即报错退出。最后，使用 listen() 开始监听传入的连接请求，最大等待队列长度设置为 10。该函数的返回值是服务端用于监听的 socket 文件描述符（serfd），将在主程序中循环用于接收客户端连接。

```
1. SOCKET listenClient()
2. {
3.     SOCKET serfd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
4.     if (serfd == INVALID_SOCKET)
5.     {
6.         error_die("socket");
7.     }
8.
9.     struct sockaddr_in serAddr;
10.    serAddr.sin_family = AF_INET;
11.    serAddr.sin_port = htons(9999);
12.    serAddr.sin_addr.S_un.S_addr = INADDR_ANY;
```

```
13.
14.     if (SOCKET_ERROR == bind(serfd, (struct sockaddr*)&serAddr, s
    izeof(serAddr)))
15.     {
16.         error_die("bind");
17.     }
18.
19.     listen(serfd, 10);
20.     return serfd;
21. }
```

（3）主程序运行与客户端连接处理

在 main() 函数中，首先调用 initSocket() 初始化网络库，然后通过 listenClient() 获取用于监听的服务端 socket（serfd）。程序打印欢迎信息后，进入一个无限循环，持续监听客户端的连接请求。每当一个新的连接通过 accept() 被接收时，将生成一个新的客户端 socket（clifd）用于与该客户端通信。

接受连接成功后，程序先通过 send() 向客户端发送 HTTP 响应头（sendData），声明返回的是 HTTP/1.1 200 OK，内容类型为 text/html（注意：你写成了 textml，应修正），接着调用 accept_request(clifd) 处理具体请求。处理完毕后，调用 closesocket(clifd) 关闭当前客户端连接，继续等待下一个连接。

```
1. void accept_request(SOCKET clifd); // 前向声明
2. void sendHtml(SOCKET clifd, const char* filePath); // 前向声明
3.
4. int main()
5. {
6.     initSocket();
7.     SOCKET serfd = listenClient();
8.
9.     printf("欢迎欢迎欢迎光临\n");
10.
11.     while (1)
12.     {
13.         struct sockaddr_in cliAddr;
14.         int len = sizeof(cliAddr);
15.         SOCKET clifd = accept(serfd, (struct sockaddr*)&cliAddr,
    &len);
16.         if (clifd == INVALID_SOCKET)
17.         {
18.             error_die("accept");
19.         }
```

```
20.
21.         // 发送简单的 HTTP 响应头（注意：textml -> text/html）
22.         const char* sendData = "HTTP/1.1 200 OK\r\nContent-Type:
    text/html\r\n\r\n";
23.         send(clifd, sendData, strlen(sendData), 0);
24.
25.         accept_request(clifd); // 处理客户端请求
26.
27.         closesocket(clifd); // 关闭连接
28.     }
29.
30.     closesocket(serfd);
31.     WSACleanup();
32.
33.     return 0;
34. }
```

（4）客户端请求解析与网页返回

客户端连接建立后，accept_request() 函数负责接收请求数据并返回网页内容。程序先通过 recv() 从客户端 socket 接收数据，存储到 recvBuf 中，如果接收失败或客户端已断开连接，则调用 error_die() 报错退出。收到的数据（通常为 HTTP GET 请求）被打印输出用于调试。

然后程序设定网页路径为 C:/Users/asus/Desktop/x/试卷卷卷子/网络实习/person/code/mail.html，并使用 _access() 函数检查该路径文件是否存在。如果存在，则调用 sendHtml(clifd, filePath) 将网页内容发送给客户端；否则打印 "NO" 提示文件不存在，可能代表返回 404 页面（目前未实现 404 处理逻辑）。

```
1. void accept_request(SOCKET clifd)
2. {
3.     char recvBuf[1024] = { 0 };
4.     int ret = recv(clifd, recvBuf, sizeof(recvBuf) - 1, 0);
5.     if (ret <= 0)
6.     {
7.         error_die("recv");
8.     }
9.
10.     printf("[client request]:\n%s\n", recvBuf);
11.
12.     const char* filePath = "C:/Users/asus/Desktop/x/试卷卷卷子/网
    络实习/person/code/mail.html";
13.     if (_access(filePath, 0) != -1)
```

```
    {
        sendHtml(clifd, filePath);
    }
    else
    {
        printf("NO\n");
        // TODO: 可以发送 404 页面
    }
}
```

（5）网页文件内容读取与发送

sendHtml() 函数用于将本地 HTML 文件的内容逐行发送给客户端。它通过 fopen() 打开指定路径的文件，并在出错时调用 error_die() 报错退出。接着使用 fgets() 从文件中逐行读取数据，调用 send() 将每一行 HTML 数据发送到客户端 socket，直到文件末尾。最后调用 fclose() 关闭文件。

这种设计使得服务端可以通过读取本地文件的方式动态返回网页内容，虽然简单但清晰地体现了基于 Socket 的 HTTP 服务原理。

```
void sendHtml(SOCKET clifd, const char* filePath)
{
    FILE* fp = fopen(filePath, "r");
    if (fp == NULL)
    {
        error_die("fopen");
    }

    char buf[1024];
    while (fgets(buf, sizeof(buf), fp))
    {
        send(clifd, buf, strlen(buf), 0);
    }

    fclose(fp);
}
```

所对应的 html 代码如下所示：

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
```

```
    <meta charset="UTF-8">
    <title>我的 Web 服务器主页</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 40px;
            background-color: #f0f0f0;
        }
        h1 {
            color: #2c3e50;
        }
        button {
            margin: 5px;
            padding: 10px 20px;
            font-size: 16px;
        }
        #result {
            margin-top: 20px;
            background: white;
            padding: 10px;
            border: 1px solid #ccc;
            white-space: pre-wrap;
        }
    </style>
</head>
<body>
    <h1>欢迎来到我的 C++ Web 服务器主页</h1>
    <p>点击下面的按钮以发送 HTTP 请求（模拟 GET / POST / PUT / DELETE / SHUTDOWN）</p>

    <button onclick="sendRequest('GET')">GET 请求</button>
    <button onclick="sendRequest('POST')">POST 请求</button>
    <button onclick="sendRequest('PUT')">PUT 请求</button>
    <button onclick="sendRequest('DELETE')">DELETE 请求</button>
    <!-- 新增关闭服务器按钮 -->
    <button onclick="shutdownServer()">关闭服务器</button>

    <div id="result">返回结果将在这里显示。</div>

    <script>
        function sendRequest(method) {
```

```
            fetch("/", {
                method: method,
                headers: {
                    'Content-Type': 'application/json'
                },
                body: method !== 'GET' ? JSON.stringify({ msg: "test_data" }) : null
            })
            .then(response => response.text())
            .then(data => {
                document.getElementById("result").innerText = method + " 返回结果:\n" + data;
            })
            .catch(error => {
                document.getElementById("result").innerText = "发生错误: " + error;
            });
        }

        // 关闭服务器的请求
        function shutdownServer() {
            fetch("/shutdown", {
                method: 'POST'
            })
            .then(response => response.text())
            .then(data => {
                document.getElementById("result").innerText = "服务器已关闭: \n" + data;
            })
            .catch(error => {
                document.getElementById("result").innerText = "关闭失败: " + error;
            });
        }
    </script>
</body>
</html>
```

4.2 系统测试

运行经过编译后的可执行文件，观察控制台输出，如图 4.1 所示，可以看到显示 success 提示，表示 WSA 服务启用、套接字创建、端口绑定等操作顺利完从。此时服务器等待客户端的连接。

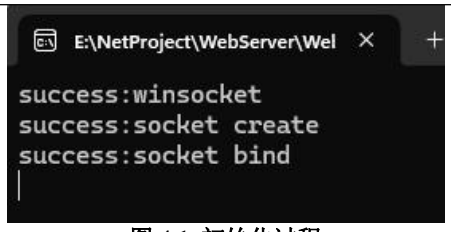


图 4.1 初始化过程

为方便测试，使用 postman 进行请求的发送，尝试发送 GET 类型的请求，观察控制台输出，注意服务开放在端口 9999，需要使用 IP:9999 的方式进行访问，控制台输出如图 4.2 所示。可以看到请求类型、URL、版本、报文头部信息均被顺利解析。

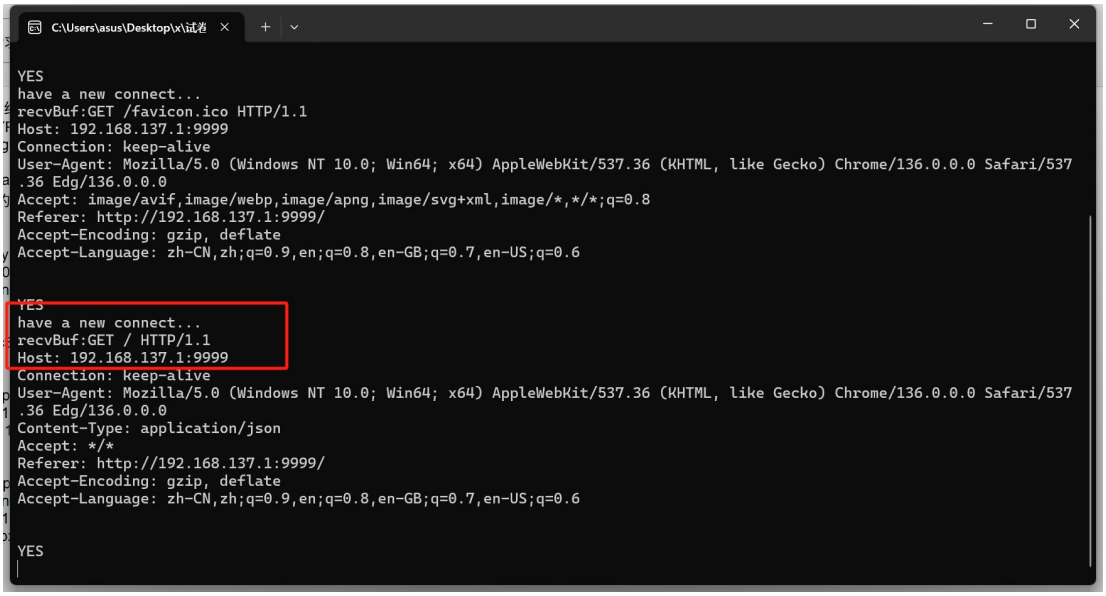


图 4.2 GET 请求的报文信息

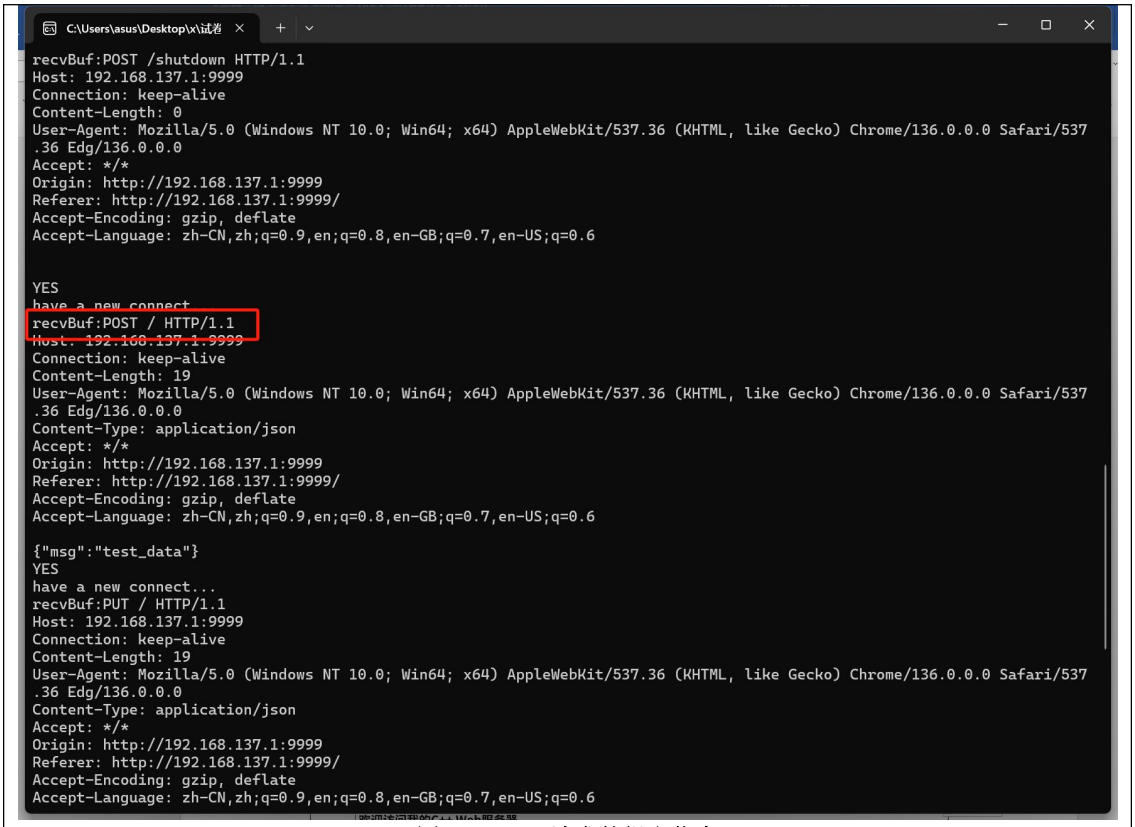


图 4.3 POST 请求的报文信息

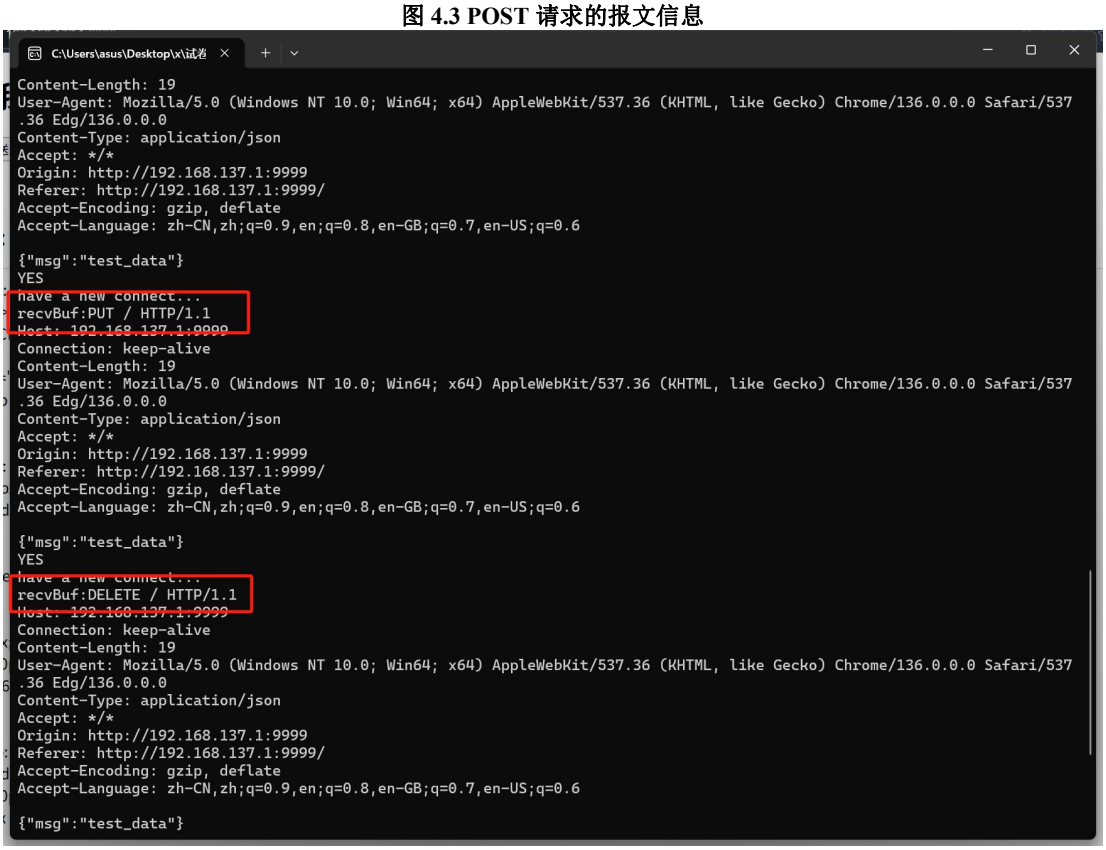


图 4.4 PUT 和 DELETE 请求的报文信息

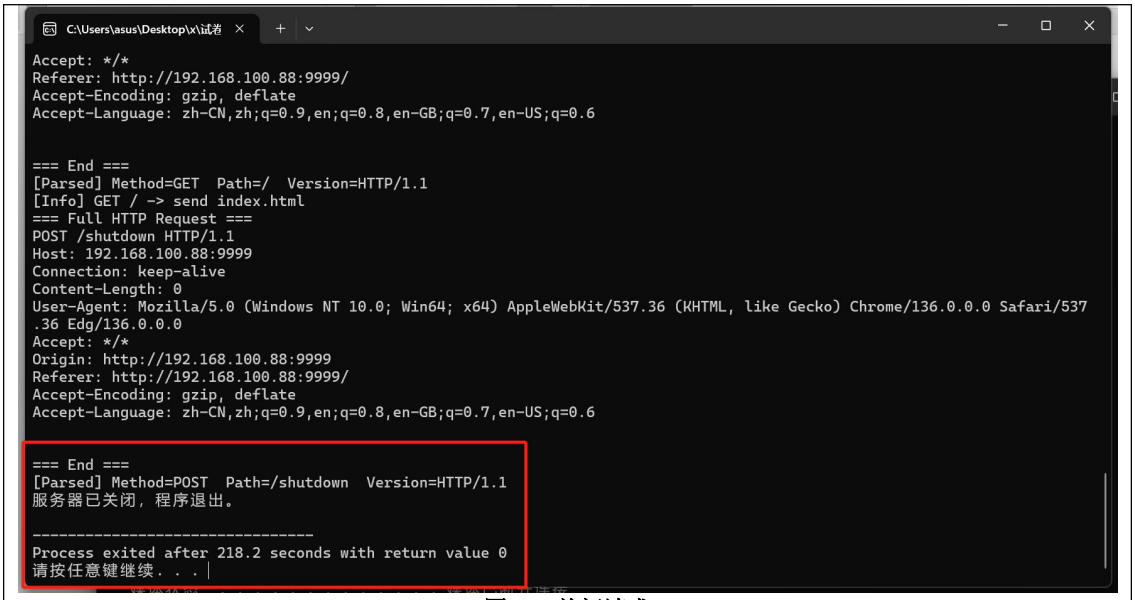


图 4.5 关闭请求

搭建的前端页面如下所示：

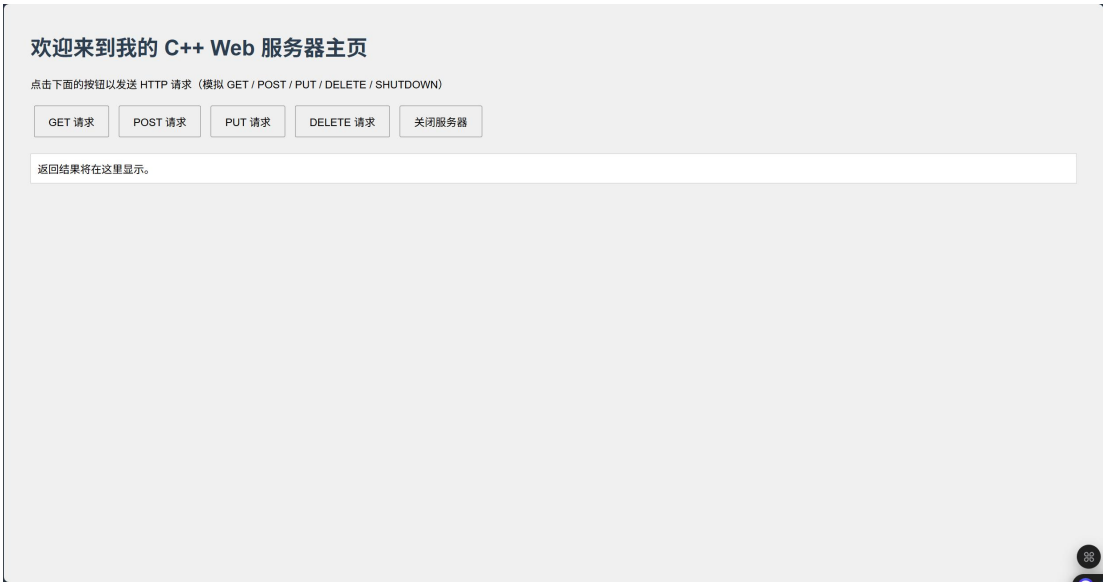


图 4.6 Web 主页

从使用对应的功能也如图所示，将会产生对应的结果返回：



图 4.7 对应不同按钮结果所示

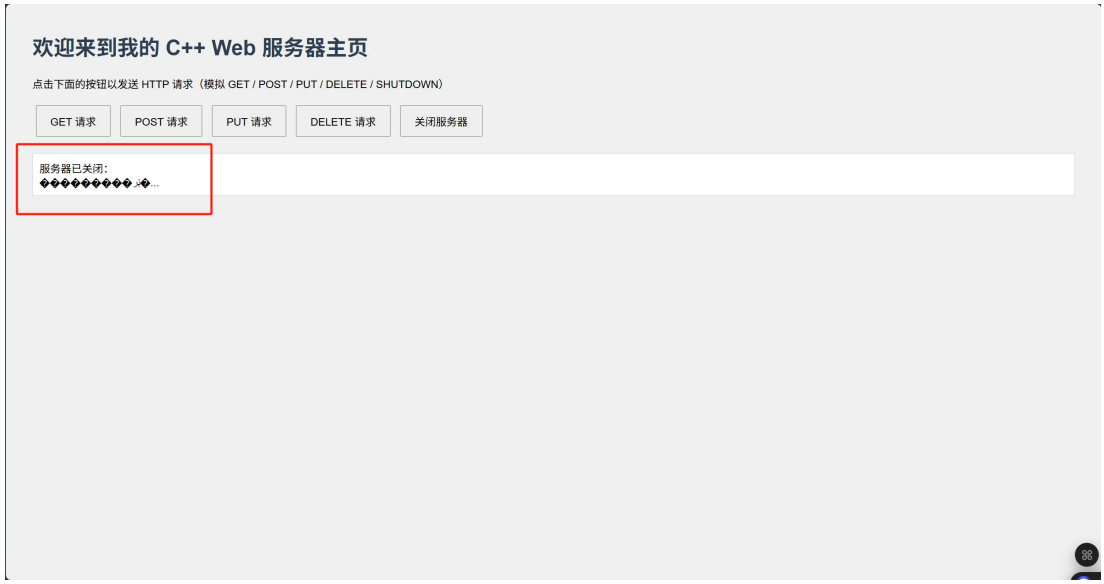


图 4.8 关闭请求结果所示

从网上随意下载一个 html 文件（以 nbu 邮箱为例），放置于与代码源文件同一路径下，HTML 内容如下。

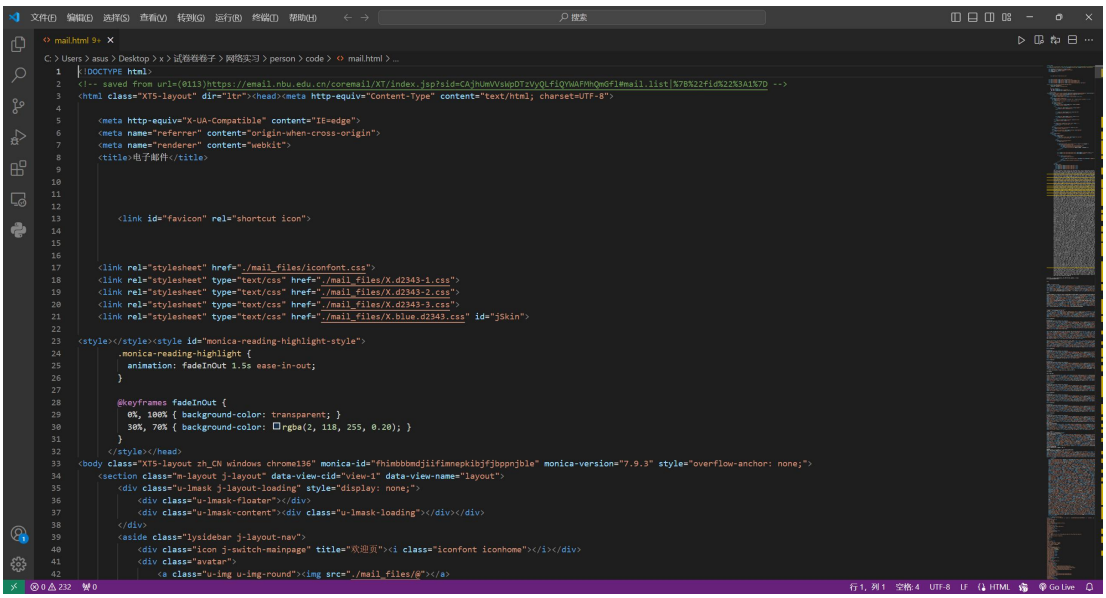


图 4.9 HTML 代码内容

使用同一子网下的主机（手机）访问运行服务器的主机 IP，需要添加服务端口 9999，结果如图 4.10 所示。可以看到成功渲染界面，可以说明 GET 请求的响应成功，能够将主页文件发送到客户端，由此判断 POST 请求得到了正确的处理。

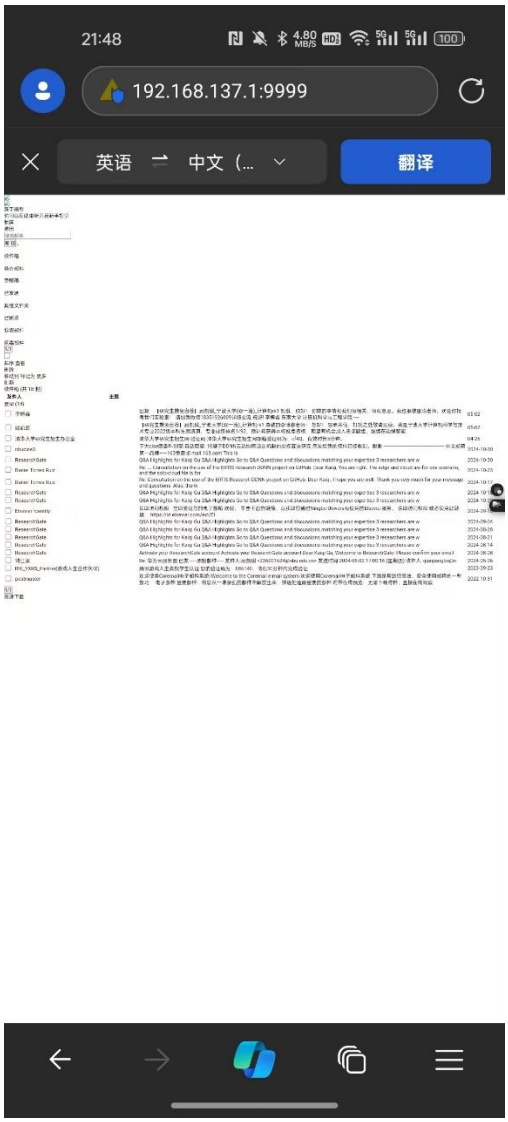


图 4.10 手机访问对应服务

五、 教师批阅意见：

成绩评定：

教师签字：

年 月 日

六、备注：